UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS – CMPF
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIA
DISCIPLINA: PROGRAMAÇÃO CONCORRENTE E DISTRIBUÍDA

QUESTÕES RESPONDIDAS UNIDADE III

FERNANDO UMBILINO ALVES

PAU DOS FERROS – RN
2025
QUESTÕES RESPONDIDAS

- QUESTÃO 36
- QUESTÃO 37
- QUESTÃO 38
- QUESTÃO 39
- QUESTÃO 40
- QUESTÃO 42
- QUESTÃO 43
- QUESTÃO 44
- QUESTÃO 45
- QUESTÃO 46
- QUESTÃO 47
- QUESTÃO 49
- QUESTÃO 50

PROGRAMAÇÃO DE MEMÓRIA DISTRIBUÍDA COM MPI

QUESTÃO 36

Modifique a regra trapezoidal para que ela estime corretamente a integral mesmo que *comm_sz* não divida *n* uniformemente. Você ainda pode assumir que $n \geq comm_sz$.

Quando o número total de trapézios (*n*) não é divisível pelo número de processos (comm_sz), a divisão (*n / comm_sz*) resultaria em um resto de trapézios não atribuídos, comprometendo a precisão da integral. Para garantir que todos os trapézios sejam considerados no cálculo, a solução foi distribuir esses trapézios restantes uniformemente entre os primeiros processos. Isso foi feito por meio de um ajuste no cálculo do *local_n* (o número de trapézios por processo), o que consequentemente impacta o *local_a* e o *local_b* (os limites do sub-intervalo de cada processo), de forma que os primeiros *n % comm_sz* ou *resto* processos recebam um trapézio adicional (`local_n = local_n + 1`).

```
C/C++
h = (b-a)/n;
local_n = (n / comm_sz);

resto = (n % comm_sz);

if(my_rank < resto){
       local_n = local_n + 1;
       local_a = a + my_rank * local_n * h;
       local_b = local_a + local_n * h;
} else {
       local_a = a + (my_rank * local_n + resto) * h;
       local_b = local_a + local_n * h;
}

local_int = Trap(local_a, local_b, local_n, h);
```

QUESTÃO 37

Modifique o programa que apenas imprime uma linha de saída de cada processo (`mpi_output.c`) para que a saída seja impressa na ordem de classificação do processo: processe a saída de 0 primeiro, depois processe 1 e assim por diante.

Foi feito a modificação para o processo 0 imprimir sua mensagem, e posteriormente receber a mensagem ordenadamente de todos os outros núcleos para imprimir na ordem.

```C/C++
const int MAX_STRING = 100;

int main(void) {

   int my_rank, comm_sz;
   char message[MAX_STRING];

   MPI_Init(NULL, NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   if(my_rank != 0){
      sprintf(message, "Proc %d of %d > Does anyone have toothpick?\n",
my_rank, comm_sz);
      MPI_Send(message, strlen(message)+1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
   } else {
      printf("Proc %d of %d > Does anyone have a toothpick?\n", my_rank,
comm_sz);
      for(int i = 1; i < comm_sz; i++){
            MPI_Recv(message, MAX_STRING, MPI_CHAR, i, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
            printf("%s", message);
      }
   }
   MPI_Finalize();

   return 0;
}  /* main */
```

Teste com `mpi_output.c` imprimindo fora de ordem:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./mpi_output
Proc 0 of 2 > Does anyone have a toothpick?
Proc 1 of 2 > Does anyone have a toothpick?
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./mpi_output
Proc 0 of 2 > Does anyone have a toothpick?
Proc 1 of 2 > Does anyone have a toothpick?
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./mpi_output
Proc 1 of 2 > Does anyone have a toothpick?
Proc 0 of 2 > Does anyone have a toothpick?
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./mpi_output
Proc 1 of 2 > Does anyone have a toothpick?
Proc 0 of 2 > Does anyone have a toothpick?
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./mpi_output
Proc 0 of 2 > Does anyone have a toothpick?
Proc 1 of 2 > Does anyone have a toothpick?
```

Teste com `mpi_output.c` modificado e imprimindo em ordem:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./q37
Proc 0 of 2 > Does anyone have a toothpick?
Proc 1 of 2 > Does anyone have toothpick?
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./q37
Proc 0 of 2 > Does anyone have a toothpick?
Proc 1 of 2 > Does anyone have toothpick?
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./q37
Proc 0 of 2 > Does anyone have a toothpick?
Proc 1 of 2 > Does anyone have toothpick?
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./q37
Proc 0 of 2 > Does anyone have a toothpick?
Proc 1 of 2 > Does anyone have toothpick?
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia$ mpiexec -n 2 ./q37
Proc 0 of 2 > Does anyone have a toothpick?
Proc 1 of 2 > Does anyone have toothpick?
```

QUESTÃO 38

Suponha que um programa seja executado com *comm_sz* processos e que *x* seja um vetor com *n* componentes. Como os componentes de *x* seriam distribuídos entre os processos em um programa que usasse uma distribuição:

(a) em bloco?

Nessa distribuição, o programa tenta dividir a quantidade de componentes de forma igual entre todos os processos. Por exemplo, se houver 4 processos e 16 componentes:

PROCESSO	COMPONENTES
0	x[0], x[1], x[2] e x[3]
1	x[4], x[5], x[6] e x[7]
2	x[8], x[9], x[10] e x[11]
3	x[12], x[13], x[14] e x[15]

Mas, quando o número total de componentes não é divisível pela quantidade de processos, os componentes restantes são distribuídos igualmente entre os primeiros processos, isso faz com que esses processos recebam um componente adicional para garantir que todo o trabalho seja coberto. Por exemplo, se houver 4 processos e 14 componentes, os 2 primeiros processos ficarão com 1 componente a mais cada, totalizando 4 componentes para cada um deles, enquanto os 2 processos restantes receberão 3 componentes cada.

(b) cíclica?

A distribuição cíclica organiza os componentes de forma sequencial, atribuindo um por um entre os processos disponíveis, ela inicia com o primeiro componente para o primeiro processo, o segundo para o segundo processo, e assim por diante. Uma vez que todos os processos receberam um componente, a atribuição retorna ao primeiro processo e continua ciclicamente até que todos os componentes estejam distribuídos. Com 4 processos e 16 componentes:

PROCESSO	COMPONENTES
0	x[0], x[4], x[8], x[12]
1	x[1], x[5], x[9], x[13]
2	x[2], x[6], x[10], x[14]
3	x[3], x[7], x[11], x[15]

(c) bloco-cíclica com tamanho de bloco *b*?

Na distribuição bloco-cíclica, os componentes são atribuídos aos processos em blocos de tamanho fixo, o primeiro bloco vai para o primeiro processo, o segundo para o segundo processo, e assim por diante. Após cada processo receber um bloco, a atribuição retorna ao primeiro processo, continuando a sequência com o próximo bloco, esse padrão continua até que todos os componentes sejam distribuídos.
Com 4 processos, 16 componentes e bloco igual a 2:

PROCESSO	COMPONENTES
0	x[0] e x[1], x[8] e x[9]
1	x[2] e x[3], x[10] e x[11]
2	x[4] e x[5], x[12] e x[13]
3	x[6] e x[7], x[14] e x[15]

Suas respostas devem ser genéricas para que possam ser usadas independentemente dos valores de comm_sz e n. Ao mesmo tempo, as distribuições apresentadas nas respostas devem ser "justas", de modo que, se q e r forem dois processos quaisquer, a diferença entre o número de componentes atribuídos a q e a r seja a menor possível.

QUESTÃO 39

Escreva um programa MPI que receba do usuário dois vetores e um escalar, todos lidos pelo processo *0* e distribuídos entre os processos. O primeiro vetor deve ser multiplicado pelo escalar. Para o segundo vetor, deve-se calcular a sua norma. Os resultados calculados devem ser coletados no processo *0*, que os imprime. Você pode assumir que *n*, a ordem dos vetores, é divisível por *comm_sz*.

```
C/C++
int main(void) {
   int my_rank, comm_sz, n_vector, local_n_vector, scale;
   double *local_vector_1, *local_vector_2;
   double sum = 0, normalization=0;

   MPI_Init(NULL, NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   Read_n(&n_vector, &local_n_vector, my_rank, comm_sz, MPI_COMM_WORLD);

   Allocate_vector(&local_vector_1, local_n_vector, MPI_COMM_WORLD);

   Read_vector(local_vector_1, local_n_vector, n_vector, "X", my_rank,
MPI_COMM_WORLD);

   Print_vector(local_vector_1, local_n_vector, n_vector, "X", my_rank,
MPI_COMM_WORLD);

   Set_scale(&scale, my_rank, MPI_COMM_WORLD);

   for(int i = 0; i < local_n_vector; i++){
      local_vector_1[i] = local_vector_1[i]*scale;
   }

   Print_vector(local_vector_1, local_n_vector, n_vector, "Vetor X
multiplicado pelo escalar", my_rank, MPI_COMM_WORLD);

   Allocate_vector(&local_vector_2, local_n_vector, MPI_COMM_WORLD);

   Read_vector(local_vector_2, local_n_vector, n_vector, "Y", my_rank,
MPI_COMM_WORLD);

   Print_vector(local_vector_2, local_n_vector, n_vector, "Y", my_rank,
MPI_COMM_WORLD);

   for(int i = 0; i < local_n_vector; i++){
      local_vector_2[i] = local_vector_2[i]*local_vector_2[i];
      sum += local_vector_2[i];
   }
```

```
   MPI_Reduce(&sum, &normalization, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);

   if(my_rank == 0){
      normalization = sqrt(normalization);
      printf("Norma do vetor Y: %.2lf\n", normalization);
   }

   free(local_vector_1);
   free(local_vector_2);
   MPI_Finalize();
   return 0;
}  /* main */

void Check_for_error(
      int       local_ok   /* in */,
      char      fname[]    /* in */,
      char      message[]  /* in */,
      MPI_Comm  comm       /* in */) {
   int ok;

   MPI_Allreduce(&local_ok, &ok, 1, MPI_INT, MPI_MIN, comm);
   if (ok == 0) {
      int my_rank;
      MPI_Comm_rank(comm, &my_rank);
      if (my_rank == 0) {
         fprintf(stderr, "Proc %d > In %s, %s\n", my_rank, fname,
               message);
         fflush(stderr);
      }
      MPI_Finalize();
      exit(-1);
   }
}  /* Check_for_error */

void Read_n(
      int*      n_p        /* out */,
      int*      local_n_p  /* out */,
      int       my_rank    /* in  */,
      int       comm_sz    /* in  */,
      MPI_Comm  comm       /* in  */) {
   int local_ok = 1;
   char *fname = "Read_n";

   if (my_rank == 0) {
      printf("Quantos elementos no vetor?\n");
      scanf("%d", n_p);
```

```
   }
   MPI_Bcast(n_p, 1, MPI_INT, 0, comm);
   if (*n_p <= 0 || *n_p % comm_sz != 0) local_ok = 0;
   Check_for_error(local_ok, fname, "n should be > 0 and evenly divisible by
comm_sz", comm);
   *local_n_p = *n_p/comm_sz;
}  /* Read_n */

void Allocate_vector(
      double**   local_x_pp  /* out */,
      int        local_n     /* in  */,
      MPI_Comm   comm        /* in  */) {
   int local_ok = 1;
   char* fname = "Allocate_vector";

   *local_x_pp = malloc(local_n*sizeof(double));

   if (*local_x_pp == NULL) local_ok = 0;
   Check_for_error(local_ok, fname, "Can't allocate local vector(s)", comm);
}  /* Allocate_vectors */

void Read_vector(
      double    local_a[]   /* out */,
      int       local_n     /* in  */,
      int       n           /* in  */,
      char      vec_name[]  /* in  */,
      int       my_rank     /* in  */,
      MPI_Comm  comm        /* in  */) {

   double* a = NULL;
   int i;
   int local_ok = 1;
   char* fname = "Read_vector";

   if (my_rank == 0) {
      a = malloc(n*sizeof(double));
      if (a == NULL) local_ok = 0;
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
comm);
      printf("Informe o vetor %s\n", vec_name);
      for (i = 0; i < n; i++){
         scanf("%lf", &a[i]);
      }

      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE, 0,
comm);
      free(a);
   } else {
```

```
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE, 0,
comm);
   }
}  /* Read_vector */

void Print_vector(
      double    local_b[]  /* in */,
      int       local_n    /* in */,
      int       n          /* in */,
      char      title[]    /* in */,
      int       my_rank    /* in */,
      MPI_Comm  comm       /* in */) {

   double* b = NULL;
   int i;
   int local_ok = 1;
   char* fname = "Print_vector";

   if (my_rank == 0) {
      b = malloc(n*sizeof(double));
      if (b == NULL) local_ok = 0;
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE,
            0, comm);
      printf("%s : ", title);
      for (i = 0; i < n; i++)
         printf("%.2f ", b[i]);
      printf("\n");
      free(b);
   } else {
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE, 0,
         comm);
   }
}  /* Print_vector */

void Set_scale(
   int* scale            /* out */,
   int my_rank          /* in */,
   MPI_Comm  comm       /* in */
){
   if (my_rank == 0) {
      printf("Informe o escalar:\n");
      scanf("%d", scale);
```

```
    }
    MPI_Bcast(scale, 1, MPI_INT, 0, comm);
}
```

Teste onde *n* é divisível por *comm_sz*, pois *n = 4* e *comm_sz = 2*:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia/q39$ mpiexec -n 2 ./q39
Quantos elementos no vetor?
4
Informe o vetor X
4 3 2 1
X : 4.00 3.00 2.00 1.00
Informe o escalar:
2
Vetor X multiplicado pelo escalar : 8.00 6.00 4.00 2.00
Informe o vetor Y
8 7 6 5
Y : 8.00 7.00 6.00 5.00
Norma do vetor Y: 13.19
```

QUESTÃO 40

Encontrar somas de prefixos é uma generalização da soma global. Em vez de simplesmente encontrar a soma de n valores,

$$x_0 + x_1 + \cdots + x_{n-1} ,$$

as somas dos prefixos são as n somas parciais

$$x_0 , x_0 + x_1 , x_0 + x_1 + x_2 , \cdots , x_0 + x_1 + \cdots + x_{n-1} .$$

(a) Elabore um algoritmo serial para calcular as n somas de prefixos de um vetor com n elementos.

```C/C++
#include <stdio.h>
#include <stdlib.h>

void Read_n(int* n_p);
void Allocate_vector(double** x_pp, int n);
void Read_vector(double a[], int n, char vec_name[]);
void Print_vector(double b[], int n, char title[]);

int main(void) {
   int n;
   double *vector_x, *vector_result;

   Read_n(&n);
   Allocate_vector(&vector_x, n);
   Read_vector(vector_x, n, "X");
   Print_vector(vector_x, n, "VETOR X:");

   Allocate_vector(&vector_result, n);

   int aux = 0;
    for (int i = 0; i < n; i++) {
        aux += vector_x[i];
        vector_result[i] = aux;
    }

   Print_vector(vector_result, n, "RESULTADO:");

   free(vector_x);
   free(vector_result);

   return 0;
}  /* main */

void Read_n(int* n_p /* out */) {
   printf("Quantos elementos no vetor?\n");
   scanf("%d", n_p);
   if (*n_p <= 0) {
```

```c
      fprintf(stderr, "Order should be positive\n");
      exit(-1);
   }
}  /* Read_n */

void Allocate_vector(
      double**  x_pp  /* out */,
      int       n     /* in  */) {
   *x_pp = malloc(n*sizeof(double));
   if (*x_pp == NULL) {
      fprintf(stderr, "Can't allocate vectors\n");
      exit(-1);
   }
}  /* Allocate_vectors */

void Read_vector(
      double  a[]         /* out */,
      int     n           /* in  */,
      char    vec_name[]  /* in  */) {
   int i;
   printf("Informe os valores %s\n", vec_name);
   for (i = 0; i < n; i++)
      scanf("%lf", &a[i]);
}  /* Read_vector */

void Print_vector(
      double  b[]     /* in */,
      int     n       /* in */,
      char    title[] /* in */) {
   int i;
   printf("%s\n", title);
   for (i = 0; i < n; i++)
      printf("%.2f ", b[i]);
   printf("\n");
}  /* Print_vector */
```

Teste feito com os elementos [1 2 5 6]:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia/q40$ ./a_vector_prefixes
Quantos elementos no vetor?
4
Informe os valores X
1 2 5 6
VETOR X:
1.00 2.00 5.00 6.00
RESULTADO:
1.00 3.00 8.00 14.00
```

(b) Paralelize seu algoritmo serial para um sistema com n processos, cada um armazenando um dos elementos de x.

- Sem utilizar *MPI_Scan*

```C/C++
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

void Read_n(int* n_p, int my_rank);
void Allocate_vector(double** x_pp, int n);
void Read_vector(double a[], int n, char vec_name[]);
void Print_vector(double b[], int n, char title[]);

int main(int argc, char* argv[]) {
   int n;
   int comm_sz, my_rank;
   double *vector_x = NULL, *vector_result = NULL;
   double local_x;
   double soma_prefixo;

   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   if (my_rank == 0) {
      Read_n(&n, my_rank);
      if (n != comm_sz) {
         fprintf(stderr, "Erro: O numero de processos (%d) deve ser igual à
total de elementos (%d).\n", comm_sz, n);
         MPI_Abort(MPI_COMM_WORLD, 1);
      }
      Allocate_vector(&vector_x, n);
      Read_vector(vector_x, n, "X");
      printf("\n");
      Print_vector(vector_x, n, "VETOR X:");
   }

   // O processo 0 envia um elemento de "X" para cada processo que guarda em
"local_x".
   MPI_Scatter(vector_x, 1, MPI_DOUBLE, &local_x, 1, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

   soma_prefixo = local_x;

   // Se não é o 0, espera receber a soma do processo anterior.
   if (my_rank > 0) {
      double receber_soma;
      MPI_Recv(&receber_soma, 1, MPI_DOUBLE, my_rank - 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
```

```
      soma_prefixo += receber_soma;
   }

   // Se não é o último, precisa enviar a soma de prefixo para o próximo.
   if (my_rank < comm_sz - 1) {
      MPI_Send(&soma_prefixo, 1, MPI_DOUBLE, my_rank + 1, 0,
MPI_COMM_WORLD);
   }

   // Junta todas as somas de prefixos de cada processo no 'vector_result'
de 0.
   if (my_rank == 0) {
      Allocate_vector(&vector_result, n);
   }
   MPI_Gather(&soma_prefixo, 1, MPI_DOUBLE, vector_result, 1, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

   if (my_rank == 0) {
      Print_vector(vector_result, n, "RESULTADO:");
      printf("\n");
      free(vector_x);
      free(vector_result);
   }

   MPI_Finalize();
   return 0;
}  /* main */

void Read_n(int* n_p /* out */, int my_rank /* in */) {
   if (my_rank == 0) {
      printf("Quantos elementos no vetor?\n");
      scanf("%d", n_p);
      if (*n_p <= 0) {
         fprintf(stderr, "Order should be positive\n");
         MPI_Abort(MPI_COMM_WORLD, 1);
      }
   }
}  /* Read_n */

void Allocate_vector(
      double** x_pp  /* out */,
      int       n     /* in  */) {
   *x_pp = malloc(n*sizeof(double));
   if (*x_pp == NULL) {
      fprintf(stderr, "Can't allocate vectors\n");
      exit(-1);
   }
}  /* Allocate_vectors */
```

```c
void Read_vector(
      double  a[]         /* out */,
      int     n           /* in  */,
      char    vec_name[]  /* in  */) {
   int i;
   printf("Enter the vector %s\n", vec_name);
   for (i = 0; i < n; i++)
      scanf("%lf", &a[i]);
}  /* Read_vector */

void Print_vector(
      double  b[]     /* in */,
      int     n       /* in */,
      char    title[] /* in */) {
   int i;
   printf("%s\n", title);
   for (i = 0; i < n; i++)
      printf("%.2f ", b[i]);
   printf("\n");
}  /* Print_vector */
```

Teste feito com os elementos [5 8]:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia/q40$ mpiexec -n 2 ./b_vector_pararel
Quantos elementos no vetor?
2
Informe os valores de X
5 8

VETOR X:
5.00 8.00
RESULTADO:
5.00 13.00
```

(c) Suponha $n = 2^k$ para algum inteiro positivo k. Crie um algoritmo paralelo que exija apenas k fases de comunicação.

- Sem utilizar *MPI_Scan*

```c
C/C++
#include <stdio.h>
#include <mpi.h>
#include <stdlib.h>
#include <math.h>

void Check_for_error(int local_ok, char fname[], char message[], MPI_Comm
comm);
void Read_n(int* k_p, int* n_p, int my_rank, int comm_sz, MPI_Comm comm);
void Allocate_vector(double** x_pp, int n, MPI_Comm comm);
```

```c
void Read_vector(double a[], int n, char vec_name[], int my_rank, MPI_Comm
comm);
void Element_prefixes(double  d[], double  c[], int n);

int main(void) {
   int k, n, comm_sz, my_rank, i, start, duo_i, factor_one = 1 , factor_two
= 2, location, displacement;
   double *vector_x, *vector_result;

   MPI_Init(NULL, NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   Read_n(&k, &n, my_rank, comm_sz, MPI_COMM_WORLD);
   Allocate_vector(&vector_x, n, MPI_COMM_WORLD);
   if(my_rank==0){
      printf("%d ELEMENTOS\n", n);
   }
   Read_vector(vector_x, n, "X", my_rank, MPI_COMM_WORLD);
   displacement = n / comm_sz;
   Allocate_vector(&vector_result, n, MPI_COMM_WORLD);

   start = my_rank*displacement;
   for(i = my_rank*displacement; i<(start+displacement); i++){
      Element_prefixes(vector_result, vector_x, i);
   }

   while (factor_one < comm_sz)
   {
      if((my_rank % factor_one) == 0){
         if((my_rank % factor_two) == 0){
            duo_i = my_rank + factor_one;
            location = start+displacement;
            MPI_Recv(&vector_result[location], displacement, MPI_DOUBLE,
duo_i, 0, MPI_COMM_WORLD, NULL);
         }else{
            duo_i = my_rank - factor_one;
            MPI_Send(&vector_result[start], displacement, MPI_DOUBLE, duo_i,
0, MPI_COMM_WORLD);
         }
      }
      factor_one *= 2;
      factor_two *= 2;
      displacement *= 2;
   }
   if(my_rank == 0){
      printf("RESULTADO: ");
      for(i = 0; i < n ; i++){
```

```c
         printf("%.2f ",vector_result[i]);
      }
      printf("\n");
   }

   free(vector_result);
   free(vector_x);
   MPI_Finalize();
   return 0;
}  /* main */

void Check_for_error(
      int       local_ok   /* in */,
      char      fname[]    /* in */,
      char      message[]  /* in */,
      MPI_Comm  comm       /* in */) {
   int ok;

   MPI_Allreduce(&local_ok, &ok, 1, MPI_INT, MPI_MIN, comm);
   if (ok == 0) {
      int my_rank;
      MPI_Comm_rank(comm, &my_rank);
      if (my_rank == 0) {
         fprintf(stderr, "Proc %d > In %s, %s\n", my_rank, fname,
               message);
         fflush(stderr);
      }
      MPI_Finalize();
      exit(-1);
   }
}  /* Check_for_error */

void Read_n(
      int*      k_p        /* out */,
      int*      n_p        /* out */,
      int       my_rank    /* in  */,
      int       comm_sz    /* in  */,
      MPI_Comm  comm       /* in  */) {
   int local_ok = 1;
   char *fname = "Read_n";

   if (my_rank == 0) {
      printf("Qual o valor de k?\n");
      scanf("%d", k_p);
      *n_p = pow(2, *k_p);
   }
   MPI_Bcast(n_p, 1, MPI_INT, 0, comm);
   if (*n_p <= 0 || *n_p % comm_sz != 0) local_ok = 0;
```

```
   Check_for_error(local_ok, fname, "n should be > 0 and evenly divisible by
comm_sz", comm);
}  /* Read_n */

void Allocate_vector(
      double**   x_pp  /* out */,
      int        n     /* in  */,
      MPI_Comm   comm  /* in  */) {
   int local_ok = 1;
   char* fname = "Allocate_vector";

   *x_pp = malloc(n*sizeof(double));

   if (*x_pp == NULL) local_ok = 0;
   Check_for_error(local_ok, fname, "Can't allocate local vector(s)", comm);
}  /* Allocate_vectors */

void Read_vector(
      double    a[]         /* out */,
      int       n           /* in  */,
      char      vec_name[]  /* in  */,
      int       my_rank     /* in  */,
      MPI_Comm  comm        /* in  */) {

   int i;
   int local_ok = 1;
   char* fname = "Read_vector";

   if (a == NULL) local_ok = 0;
   Check_for_error(local_ok, fname, "Vector not allocated", comm);

   if (my_rank == 0) {
      printf("Informe o vetor %s\n", vec_name);
      for (i = 0; i < n; i++){
         scanf("%lf", &a[i]);
      }
   }
   MPI_Bcast(a, n, MPI_DOUBLE, 0, comm);
}  /* Read_vector */

void Element_prefixes(
      double  d[]     /* out */,
      double  c[]     /* in */,
      int     n       /* in */) {
   d[n] = 0;
   for(int i = 0; i <= n; i++){
      d[n] += c[i];
   }
```

```
}
```

(d) O MPI fornece uma função de comunicação coletiva, *MPI_Scan*, que pode ser usada para calcular somas de prefixos:

```
int MPI Scan(
void* sendbuf p /* in */,
void* recvbuf p /* out */,
int count /* in */,
MPI Datatype datatype /* in */,
MPI Op op /* in */,
MPI Comm comm /* in */);
```

Ela opera em arrays com *count* elementos; *sendbuf_p* e *recvbuf_p* devem se referir a blocos de *count* elementos do tipo *datatype*. O argumento *op* é igual ao *op* para o *MPI_Reduce*. Escreva um programa MPI que gere um vetor aleatório de *count* elementos em cada processo MPI, encontre as somas dos prefixos e imprima os resultados.

```C/C++
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <mpi.h>

void print_vector(const char* titulo, int* vetor, int tamanho) {
    printf("%s: [ ", titulo);
    for (int i = 0; i < tamanho; i++) {
        printf("%d ", vetor[i]);
    }
    printf("]\n");
}

int main(int argc, char* argv[]) {
    const int COUNT = 5; //elementos em cada vetor
    int meu_rank, comm_sz;
    int *vetor_local = NULL;
    int *somas_de_prefixo = NULL;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_rank);

    vetor_local = malloc(COUNT * sizeof(int));
```

```c
    somas_de_prefixo = malloc(COUNT * sizeof(int));

    if (vetor_local == NULL || somas_de_prefixo == NULL) {
        fprintf(stderr, "Processo %d: Nao alocou memoria!\n", meu_rank);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    srand(time(NULL) + meu_rank);
    for (int i = 0; i < COUNT; i++) {
        vetor_local[i] = rand() % 10;
    }

    MPI_Scan(vetor_local, somas_de_prefixo, COUNT, MPI_INT, MPI_SUM,
MPI_COMM_WORLD);

    for (int i = 0; i < comm_sz; i++) {
        if (meu_rank == i) {
            printf("--- Processo %d ---\n", meu_rank);
            print_vector("Vetor gerado", vetor_local, COUNT);
            print_vector("Somas de Prefixo", somas_de_prefixo, COUNT);
            printf("\n");
        }
        MPI_Barrier(MPI_COMM_WORLD);
    }

    free(vetor_local);
    free(somas_de_prefixo);

    MPI_Finalize();
    return 0;
}
```

Saida do terminal:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia/q40$ mpiexec -n 2 ./d_mpiScan_vector
--- Processo 0 ---
Vetor gerado: [ 2 3 5 4 8 ]
Somas de Prefixo: [ 2 3 5 4 8 ]

--- Processo 1 ---
Vetor gerado: [ 7 8 3 1 7 ]
Somas de Prefixo: [ 9 11 8 5 15 ]
```

QUESTÃO 41

Uma alternativa para um *allreduce* estruturado em borboleta é uma estrutura de passagem em anel. Em uma passagem de anel, se houver p processos, cada processo q envia dados para o processo $q + 1$, exceto que o processo $p - 1$ envia dados para o processo 0. Isso é repetido até que cada processo tenha o resultado desejado. Assim, podemos implementar *allreduce* com o seguinte código:

```
sum = temp_val = my_val;
for (i = 1; i < p; i++) {
MPI_Sendrecv_replace(&temp_val, 1, MPI INT, dest,
                     sendtag, source, recvtag, comm, &status);
      sum += temp_val;
}
```

(a) Escreva um programa MPI que implemente esse algoritmo para o *allreduce*. Como seu desempenho se compara ao *allreduce* estruturado em borboleta?

(b) Modifique o programa MPI que você escreveu na primeira parte para que ele implemente somas de prefixos.

QUESTÃO 42

As funções *MPI_Scatter* e *MPI_Gather* têm a limitação de que cada processo deve enviar ou receber o mesmo número de itens de dados. Quando este não for o caso, devemos utilizar as funções *MPI_Gatherv* e *MPI_Scatterv*. Consulte a documentação dessas funções e modifique seu programa da questão **39** para que ele possa lidar corretamente com o caso quando *n* não é divisível por *comm_sz*.

```
C/C++
int main(void) {
   int my_rank, comm_sz, n_vector, local_n_vector, scale, *vector_send,
*vector_display;
   double *local_vector_1, *local_vector_2;
   double sum = 0, normalization=0;

   MPI_Init(NULL, NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   Read_n(&n_vector, &local_n_vector, my_rank, comm_sz, MPI_COMM_WORLD);

   Allocate_vector(&local_vector_1, local_n_vector, MPI_COMM_WORLD);

   if (my_rank == 0) {
       vector_send = malloc(comm_sz * sizeof(int));
       vector_display = malloc(comm_sz * sizeof(int));
   }

   MPI_Gather(&local_n_vector, 1, MPI_INT, vector_send, 1, MPI_INT, 0,
MPI_COMM_WORLD);

   if (my_rank == 0){
      vector_display[0] = 0;
      for(int i = 0; i < comm_sz-1; i++){
         vector_display[i+1] = vector_display[i] + vector_send[i];
      }
   }

   Read_vector(local_vector_1, vector_send, vector_display, local_n_vector,
n_vector, "X", my_rank, MPI_COMM_WORLD);

   Print_vector(local_vector_1, vector_send, vector_display, local_n_vector,
n_vector, "X", my_rank, MPI_COMM_WORLD);

   Set_scale(&scale, my_rank, MPI_COMM_WORLD);

   for(int i = 0; i < local_n_vector; i++){
      local_vector_1[i] = local_vector_1[i]*scale;
   }
```

```
   Print_vector(local_vector_1, vector_send, vector_display, local_n_vector,
n_vector, "Vetor X multiplicado pelo escalar", my_rank, MPI_COMM_WORLD);

   Allocate_vector(&local_vector_2, local_n_vector, MPI_COMM_WORLD);

   Read_vector(local_vector_2, vector_send, vector_display, local_n_vector,
n_vector, "Y", my_rank, MPI_COMM_WORLD);

   Print_vector(local_vector_2, vector_send, vector_display, local_n_vector,
n_vector, "Y", my_rank, MPI_COMM_WORLD);

   for(int i = 0; i < local_n_vector; i++){
      local_vector_2[i] = local_vector_2[i]*local_vector_2[i];
      sum += local_vector_2[i];
   }

   MPI_Reduce(&sum, &normalization, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);

   if(my_rank == 0){
      normalization = sqrt(normalization);
      printf("Norma do vetor Y: %.2lf\n", normalization);
   }

   free(local_vector_1);
   free(local_vector_2);
   if(my_rank==0){
      free(vector_send);
   }
   MPI_Finalize();
   return 0;
}  /* main */

void Check_for_error(
      int       local_ok   /* in */,
      char      fname[]    /* in */,
      char      message[]  /* in */,
      MPI_Comm  comm       /* in */) {
   int ok;

   MPI_Allreduce(&local_ok, &ok, 1, MPI_INT, MPI_MIN, comm);
   if (ok == 0) {
      int my_rank;
      MPI_Comm_rank(comm, &my_rank);
      if (my_rank == 0) {
         fprintf(stderr, "Proc %d > In %s, %s\n", my_rank, fname,
               message);
```

```
         fflush(stderr);
      }
      MPI_Finalize();
      exit(-1);
   }
}  /* Check_for_error */

void Read_n(
      int*      n_p        /* out */,
      int*      local_n_p  /* out */,
      int       my_rank    /* in  */,
      int       comm_sz    /* in  */,
      MPI_Comm  comm       /* in  */) {
   int local_ok = 1, resto;
   char *fname = "Read_n";

   if (my_rank == 0) {
      printf("Quantos elementos no vetor?\n");
      scanf("%d", n_p);
   }
   MPI_Bcast(n_p, 1, MPI_INT, 0, comm);
   if (*n_p <= 0) local_ok = 0;
   Check_for_error(local_ok, fname, "n should be > 0 and evenly divisible by
comm_sz", comm);
   resto = *n_p % comm_sz;
   if(resto == 0){
      *local_n_p = *n_p/comm_sz;
   }else{
      if(my_rank<resto){
         *local_n_p = (*n_p/comm_sz) +1;
      }else{
         *local_n_p = *n_p/comm_sz;
      }
   }
}  /* Read_n */

void Allocate_vector(
      double**   local_x_pp  /* out */,
      int        local_n     /* in  */,
      MPI_Comm   comm        /* in  */) {
   int local_ok = 1;
   char* fname = "Allocate_vector";

   *local_x_pp = malloc(local_n*sizeof(double));

   if (*local_x_pp == NULL) local_ok = 0;
   Check_for_error(local_ok, fname, "Can't allocate local vector(s)", comm);
}  /* Allocate_vectors */
```

```c
void Read_vector(
      double    local_a[]   /* out */,
      int       sendcounts[] ,
      int       display[],
      int       local_n     /* in  */,
      int       n           /* in  */,
      char      vec_name[]  /* in  */,
      int       my_rank     /* in  */,
      MPI_Comm  comm        /* in  */) {

   double* a = NULL;
   int i;
   int local_ok = 1;
   char* fname = "Read_vector";

   if (my_rank == 0) {
      a = malloc(n*sizeof(double));
      if (a == NULL) local_ok = 0;
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
comm);
      printf("Informe o vetor %s\n", vec_name);
      for (i = 0; i < n; i++){
         scanf("%lf", &a[i]);
      }
      MPI_Scatterv(a, sendcounts, display, MPI_DOUBLE, local_a, local_n,
MPI_DOUBLE, 0, comm);
      free(a);
   } else {
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Scatterv(a, sendcounts, display, MPI_DOUBLE, local_a, local_n,
MPI_DOUBLE, 0, comm);
   }
}  /* Read_vector */

void Print_vector(
      double    local_b[]  /* in */,
      int       recvcounts[]/* in */,
      int       displs[]   /* in */,
      int       local_n    /* in */,
      int       n          /* in */,
      char      title[]    /* in */,
      int       my_rank    /* in */,
      MPI_Comm comm) {

   double* b = NULL;
   int i;
```

```c
   int local_ok = 1;
   char* fname = "Print_vector";

   if (my_rank == 0) {
      b = malloc(n*sizeof(double));
      if (b == NULL) local_ok = 0;
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Gatherv(local_b, local_n, MPI_DOUBLE, b, recvcounts, displs,
MPI_DOUBLE, 0, comm);
      printf("%s: ", title);
      for (i = 0; i < n; i++)
         printf("%.2f ", b[i]);
      printf("\n");
      free(b);
   } else {
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Gatherv(local_b, local_n, MPI_DOUBLE, b, recvcounts, displs,
MPI_DOUBLE, 0, comm);
   }
}  /* Print_vector */

void Set_scale(
   int* scale            /* out */,
   int my_rank          /* in */,
   MPI_Comm  comm       /* in */
){
   if (my_rank == 0) {
      printf("Informe o escalar:\n");
      scanf("%d", scale);
   }
   MPI_Bcast(scale, 1, MPI_INT, 0, comm);
}
```

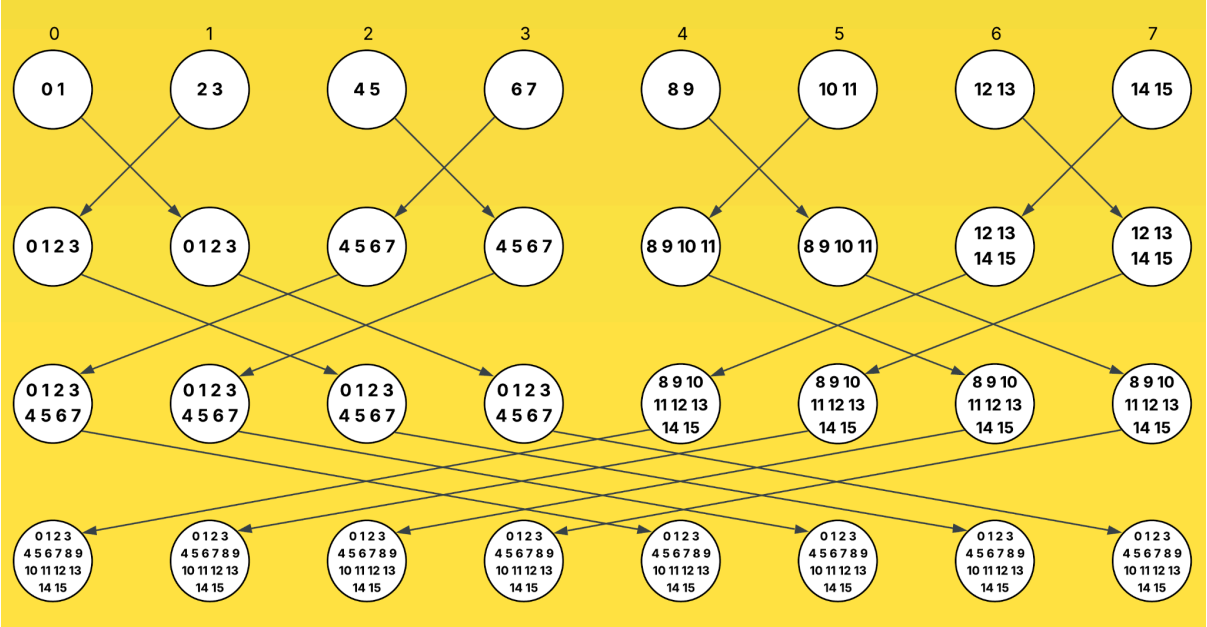
Teste onde *n* não é divisível por *comm_sz*, pois *n = 5* e *comm_sz = 2*:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia/q42$ mpiexec -n 2 ./q42
Quantos elementos no vetor?
5
Informe o vetor X
5 4 3 2 1
X: 5.00 4.00 3.00 2.00 1.00
Informe o escalar:
2
Vetor X multiplicado pelo escalar: 10.00 8.00 6.00 4.00 2.00
Informe o vetor Y
1 2 3 4 5
Y: 1.00 2.00 3.00 4.00 5.00
Norma do vetor Y: 7.42
```

QUESTÃO 43

Suponha que *comm_sz* = *8* e o vetor *x* = *(0, 1, 2, · · · , 15)* tenha sido distribuído entre os processos usando uma distribuição em bloco. Desenhe um diagrama ilustrando as etapas de uma implementação borboleta da função *allgather* de *x*.

Para uma quantidade de *8 processos* e o vetor *x* com *16* posições, uma implementação borboleta da função *MPI_Allgather*, como ilustra o diagrama, distribui eficientemente os dados de todos os processos para todos os outros em etapas. A cada etapa, os processos são organizados em pares com parceiros a uma distância que dobra exponencialmente (primeiro distância 1, depois 2, e por fim 4). Nessas trocas, cada processo envia os dados que acumulou até o momento e recebe os dados de seu parceiro, concatenando-os para formar um bloco de informações cada vez maior. Após $log_2(p)$ etapas ($\log_2(8) = 3$), onde p é o número de processos, cada processo terá acumulado os blocos de dados de todos os outros, resultando em uma cópia idêntica e completa do vetor.



QUESTÃO 44

A função *MPI_Type_contiguous* pode ser usada para construir um tipo de dados derivado de uma coleção de elementos contíguos em uma matriz. Sua sintaxe é

```
int MPI_Type_contiguous(
       int count /* in */,
       MPI Datatype old_mpi_t /* in */,
       MPI Datatype* new_mpi_t_p /* out *);
```

Modifique as funções *Read_vector* e *Print_vector (mpi_vector_add.c)* para que elas usem um tipo de dados MPI criado por uma chamada para *MPI_Type_contiguous* e um argumento de contagem de *1* nas chamadas para *MPI_Scatter* e *MPI_Gather*.

```c
C/C++
int main(void) {
   int n, local_n;
   int comm_sz, my_rank;
   double *local_x, *local_y, *local_z;
   MPI_Comm comm;

   MPI_Init(NULL, NULL);
   comm = MPI_COMM_WORLD;
   MPI_Comm_size(comm, &comm_sz);
   MPI_Comm_rank(comm, &my_rank);

   Read_n(&n, &local_n, my_rank, comm_sz, comm);

   Allocate_vectors(&local_x, &local_y, &local_z, local_n, comm);

   MPI_Datatype BLOCK;
   MPI_Type_contiguous(local_n, MPI_DOUBLE, &BLOCK);
   MPI_Type_commit(&BLOCK);

   Read_vector(local_x, local_n, n, "X", my_rank, comm, BLOCK);
   Read_vector(local_y, local_n, n, "Y", my_rank, comm, BLOCK);
   Print_vector(local_x, local_n, n, "X ", my_rank, comm, BLOCK);
   Print_vector(local_y, local_n, n, "Y ", my_rank, comm, BLOCK);

   Parallel_vector_sum(local_x, local_y, local_z, local_n);
   Print_vector(local_z, local_n, n, "SOMA", my_rank, comm, BLOCK);


   MPI_Type_free(&BLOCK);
   free(local_x);
   free(local_y);
   free(local_z);
```

```c
   MPI_Finalize();

   return 0;
}  /* main */

void Check_for_error(
      int       local_ok   /* in */,
      char      fname[]    /* in */,
      char      message[]  /* in */,
      MPI_Comm  comm       /* in */) {
   int ok;

   MPI_Allreduce(&local_ok, &ok, 1, MPI_INT, MPI_MIN, comm);
   if (ok == 0) {
      int my_rank;
      MPI_Comm_rank(comm, &my_rank);
      if (my_rank == 0) {
         fprintf(stderr, "Proc %d > In %s, %s\n", my_rank, fname,
               message);
         fflush(stderr);
      }
      MPI_Finalize();
      exit(-1);
   }
}  /* Check_for_error */

void Read_n(
      int*      n_p        /* out */,
      int*      local_n_p  /* out */,
      int       my_rank    /* in  */,
      int       comm_sz    /* in  */,
      MPI_Comm  comm       /* in  */) {
   int local_ok = 1;
   char *fname = "Read_n";

   if (my_rank == 0) {
      printf("Quantos elementos no vetor?\n");
      scanf("%d", n_p);
   }
   MPI_Bcast(n_p, 1, MPI_INT, 0, comm);
   if (*n_p <= 0 || *n_p % comm_sz != 0) local_ok = 0;
   Check_for_error(local_ok, fname,
         "n should be > 0 and evenly divisible by comm_sz", comm);
   *local_n_p = *n_p/comm_sz;
}  /* Read_n */

void Allocate_vectors(
      double**   local_x_pp  /* out */,
```

```
      double**   local_y_pp  /* out */,
      double**   local_z_pp  /* out */,
      int        local_n     /* in  */,
      MPI_Comm   comm        /* in  */) {
   int local_ok = 1;
   char* fname = "Allocate_vectors";

   *local_x_pp = malloc(local_n*sizeof(double));
   *local_y_pp = malloc(local_n*sizeof(double));
   *local_z_pp = malloc(local_n*sizeof(double));

   if (*local_x_pp == NULL || *local_y_pp == NULL ||
       *local_z_pp == NULL) local_ok = 0;
   Check_for_error(local_ok, fname, "Can't allocate local vector(s)",
         comm);
}  /* Allocate_vectors */

void Read_vector(
      double    local_a[]   /* out */,
      int       local_n     /* in  */,
      int       n           /* in  */,
      char      vec_name[]  /* in  */,
      int       my_rank     /* in  */,
      MPI_Comm  comm        /* in  */,
      MPI_Datatype BLOCK    /* in  */) {

   double* a = NULL;
   int i;
   int local_ok = 1;
   char* fname = "Read_vector";

   if (my_rank == 0) {
      a = malloc(n*sizeof(double));
      if (a == NULL) local_ok = 0;
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      printf("Informe o vetor %s\n", vec_name);
      for (i = 0; i < n; i++)
         scanf("%lf", &a[i]);
      MPI_Scatter(a, 1, BLOCK, local_a, 1, BLOCK, 0,
         comm);
      free(a);
   } else {
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Scatter(a, 1, BLOCK, local_a, 1, BLOCK, 0,
         comm);
   }
```

```c
}  /* Read_vector */

void Print_vector(
      double    local_b[]  /* in */,
      int       local_n    /* in */,
      int       n          /* in */,
      char      title[]    /* in */,
      int       my_rank    /* in */,
      MPI_Comm  comm       /* in */,
      MPI_Datatype BLOCK   /* in  */) {

   double* b = NULL;
   int i;
   int local_ok = 1;
   char* fname = "Print_vector";

   if (my_rank == 0) {
      b = malloc(n*sizeof(double));
      if (b == NULL) local_ok = 0;
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Gather(local_b, 1, BLOCK, b, 1, BLOCK,
            0, comm);
      printf("%s: ", title);
      for (i = 0; i < n; i++)
         printf("%.2f ", b[i]);
      printf("\n");
      free(b);
   } else {
      Check_for_error(local_ok, fname, "Can't allocate temporary vector",
            comm);
      MPI_Gather(local_b, 1, BLOCK, b, 1, BLOCK, 0,
         comm);
   }
}  /* Print_vector */

void Parallel_vector_sum(
      double  local_x[]  /* in  */,
      double  local_y[]  /* in  */,
      double  local_z[]  /* out */,
      int     local_n    /* in  */) {
   int local_i;

   for (local_i = 0; local_i < local_n; local_i++)
      local_z[local_i] = local_x[local_i] + local_y[local_i];
}  /* Parallel_vector_sum */
```

Teste com (*mpi_vector_add.c*) modificado:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia/q44$ mpiexec -n 2 ./q44
Quantos elementos no vetor?
4
Informe o vetor X
1 2 3 4
Informe o vetor Y
5 6 7 8
X : 1.00 2.00 3.00 4.00
Y : 5.00 6.00 7.00 8.00
SOMA: 6.00 8.00 10.00 12.00
```

QUESTÃO 45

A função *MPI_Type_indexed* pode ser usada para construir um tipo de dados derivado de elementos arbitrários de um vetor. Sua sintaxe é

```
int MPI_Type_indexed(
      int count /* in */,
      int array_of_blocklengths[] /* in */,
      int array_of_displacements[] /* in */,
      MPI_Datatype old_mpi_t /* in */,
      MPI_Datatype* new_mpi_t_p /* out */);
```

Ao contrário da função *MPI_Type_create_struct*, os deslocamentos são medidos em unidades de *old_mpi_t* - não em bytes. Use a função *MPI_Type_indexed* para criar um tipo de dados derivado que corresponda à parte triangular superior de uma matriz quadrada.
Por exemplo, na matriz *4 x 4*

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

a parte triangular superior são os elementos *0, 1, 2, 3, 5, 6, 7, 10, 11, 15*. O processo *0* deve ler uma matriz $n \times n$ como um vetor unidimensional, criar o tipo de dados derivado e enviar a parte triangular superior com uma única chamada de *MPI_Send*. O processo *1* deve receber a parte triangular superior com uma única chamada ao *MPI_Recv* e depois imprimir os dados recebidos.

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

void Check_for_error(int local_ok, char fname[], char message[],MPI_Comm
comm);
void Read_n(int* n_p, int my_rank, int comm_sz, MPI_Comm comm);
void Print_matrix( double *matrix, int n);
void Print_matrix_up( double *matrix, int n);

int main(void) {
      int n, comm_sz, my_rank, *blocklengths, *displacements;
      double *matrix, *matrix_tri_up;
      MPI_Comm comm;

      MPI_Init(NULL, NULL);
      comm = MPI_COMM_WORLD;
```

```
	MPI_Comm_size(comm, &comm_sz);
	MPI_Comm_rank(comm, &my_rank);

	Read_n(&n, my_rank, comm_sz, comm);
	blocklengths = malloc(n * sizeof(double));
	displacements = malloc(n * sizeof(double));
	for (int i = 0; i < n; i++) {
		blocklengths[i] = n - i;
		displacements[i] = i * n + i;
	}

	MPI_Datatype upper_triangle_type;
	MPI_Type_indexed(n, blocklengths, displacements, MPI_DOUBLE,
&upper_triangle_type);
	MPI_Type_commit(&upper_triangle_type);


	if (my_rank == 0) {
		matrix = malloc(n * n * sizeof(double));

		for (int i = 0; i < n; i++) {
			for (int j = 0; j < n; j++) {
			matrix[i * n + j] = rand()%100;
			}
		}

		printf("MATRIZ:\n");
		Print_matrix(matrix, n);

		MPI_Send(matrix, 1, upper_triangle_type, 1, 0, MPI_COMM_WORLD);

	} else if (my_rank == 1) {
		int num_elements = n * (n + 1) / 2;
		matrix_tri_up = malloc(num_elements * sizeof(double));

		MPI_Recv(matrix_tri_up, num_elements, MPI_DOUBLE, 0, 0,
MPI_COMM_WORLD, NULL);

		printf("\nMATRIZ TRIANGULAR:\n");

		Print_matrix_up(matrix_tri_up, n);

		free(matrix_tri_up);
	}

	if (matrix){
		free(matrix);
	}
```

```c
	free(blocklengths);
	free(displacements);
	MPI_Type_free(&upper_triangle_type);
	MPI_Finalize();
	return 0;
}

void Check_for_error(
	int       local_ok   /* in */,
	char      fname[]    /* in */,
	char      message[]  /* in */,
	MPI_Comm  comm       /* in */) {
	int ok;

	MPI_Allreduce(&local_ok, &ok, 1, MPI_INT, MPI_MIN, comm);
	if (ok == 0) {
		int my_rank;
		MPI_Comm_rank(comm, &my_rank);
		if (my_rank == 0) {
		fprintf(stderr, "Proc %d > In %s, %s\n", my_rank, fname,
			message);
		fflush(stderr);
		}
		MPI_Finalize();
		exit(-1);
	}
}  /* Check_for_error */

void Read_n(
	int*      n_p        /* out */,
	int       my_rank    /* in  */,
	int       comm_sz    /* in  */,
	MPI_Comm  comm       /* in  */) {
	int local_ok = 1;
	char *fname = "Read_n";

	if (my_rank == 0) {
		printf("Qual a ordem da Matriz?\n");
		scanf("%d", n_p);
	}
	MPI_Bcast(n_p, 1, MPI_INT, 0, comm);
	if (*n_p <= 0 || *n_p % comm_sz != 0) local_ok = 0;
	Check_for_error(local_ok, fname,
		"n should be > 0 and evenly divisible by comm_sz", comm);
}  /* Read_n */

void Print_matrix(
	double *matrix   /*  in  */,
```

```c
        int n            /*  in  */) {
        for (int i = 0; i < n; i++) {
              for (int j = 0; j < n; j++){
                    printf("%.2f\t", matrix[i * n + j]);
              }
              printf("\n");
        }
} /* Print_matrix */

void Print_matrix_up(
        double *matrix_up   /*  in  */,
        int n            /*  in  */) {

        int k = 0;
        for (int i = 0; i < n; i++) {
              for (int j = 0; j < n; j++) {
                    if (j < i){
                          printf("-\t");
                    } else {
                          printf("%.2f\t", matrix_up[k++]);
                          }
                    }
              printf("\n");
              }
} /* Print_matrix_up */
```

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia/q45$ mpiexec -n 2 ./q45
Qual a ordem da Matriz?
4
MATRIZ:
83.00   86.00   77.00   15.00
93.00   35.00   86.00   92.00
49.00   21.00   62.00   27.00
90.00   59.00   63.00   26.00

MATRIZ TRIANGULAR:
83.00   86.00   77.00   15.00
-       35.00   86.00   92.00
-       -       62.00   27.00
-       -       -       26.00
```

QUESTÃO 46

As funções *MPI_Pack* e *MPI_Unpack* fornecem uma alternativa aos tipos de dados derivados para agrupar dados. O *MPI_Pack* copia os dados a serem enviados, um bloco por vez, em um *buffer* fornecido pelo usuário. O *buffer* pode então ser enviado e recebido. Após o recebimento dos dados, *MPI_Unpack* pode ser usado para descompactá-los do *buffer* de recebimento. A sintaxe do *MPI_Pack* é

```
int MPI_Pack(
       void* in_buf /* in */,
       int in_buf_count /* in */,
       MPI_Datatype datatype /* in */,
       void* pack_buf /* out */,
       int pack_buf_sz /* in */,
       int* position_p /* in/out */,
       MPI_Comm comm /* in */);
```

Poderíamos, portanto, empacotar os dados de entrada para o programa da regra dos trapézios com o seguinte código:

```
char pack_buf[100];
int position = 0;
MPI_Pack(&a, 1, MPI_DOUBLE, pack_buf, 100, &position, comm);
MPI_Pack(&b, 1, MPI_DOUBLE, pack_buf, 100, &position, comm);
MPI_Pack(&n, 1, MPI_INT, pack_buf, 100, &position, comm);
```

A chave é o argumento da *position*. Quando *MPI_Pack* é chamado, a posição deve referir-se ao primeiro slot disponível no *pack_buf*. Quando *MPI_Pack* retorna, ele se refere ao primeiro slot disponível após os dados que acabaram de ser compactados, portanto, após o processo *0* executar este código, todos os processos podem chamar *MPI_Bcast*:

```
MPI_Bcast(pack_buf, 100, MPI_PACKED, 0, comm);
```

Observe que o tipo de dados MPI para um *buffer* compactado é *MPI_PACKED*. Agora os outros processos podem descompactar os dados usando: *MPI_Unpack*:

```
int MPI Unpack(
       void* pack_buf /* in */,
       int pack_buf_sz /* in */,
       int* position_p /* in/out */,
       void* out_buf /* out */,
       int out_buf_count /* in */,
       MPI_Datatype datatype /* in */,
       MPI_Comm comm /* in */);
```

MPI_Unpack pode ser usado "invertendo"as etapas do *MPI_Pack*, ou seja, os dados são descompactados um bloco por vez, começando em *position = 0*.

Escreva outra função *Get_input* para o programa da regra dos trapézios. Este deve usar *MPI_Pack* no processo *0* e *MPI_Unpack* nos demais processos.

```C/C++
void Get_input(int my_rank, int comm_sz, double* a_p, double* b_p, int* n_p)
{
   int size_a, size_b, size_n, total_size;
   int position = 0;
   char* pack_buf;

   MPI_Pack_size(1, MPI_DOUBLE, MPI_COMM_WORLD, &size_a);
   MPI_Pack_size(1, MPI_DOUBLE, MPI_COMM_WORLD, &size_b);
   MPI_Pack_size(1, MPI_INT,    MPI_COMM_WORLD, &size_n);

   total_size = size_a + size_b + size_n;
   pack_buf = (char*) malloc(total_size);

   if (my_rank == 0) {

      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);

      MPI_Pack(a_p, 1, MPI_DOUBLE, pack_buf, total_size, &position,
MPI_COMM_WORLD);
      MPI_Pack(b_p, 1, MPI_DOUBLE, pack_buf, total_size, &position,
MPI_COMM_WORLD);
      MPI_Pack(n_p, 1, MPI_INT, pack_buf, total_size, &position,
MPI_COMM_WORLD);
   }
   MPI_Bcast(pack_buf, total_size, MPI_PACKED, 0, MPI_COMM_WORLD);

   position = 0;

   MPI_Unpack(pack_buf, total_size, &position, a_p, 1, MPI_DOUBLE,
MPI_COMM_WORLD);
   MPI_Unpack(pack_buf, total_size, &position, b_p, 1, MPI_DOUBLE,
MPI_COMM_WORLD);
   MPI_Unpack(pack_buf, total_size, &position, n_p, 1, MPI_INT,
MPI_COMM_WORLD);

   free(pack_buf);
}  /* Get_input */
```

Resultado após execução:

```
fernando@ubuntu:~/Documentos/PCD/ipp-source-copia/ipp-source-use/ch3-copia/q46$ mpiexec -n 2 ./q46
Enter a, b, and n
10 52 100
With n = 100 trapezoids, our estimate
of the integral from 10.000000 to 52.000000 = 4.653723480000000e+04
```

QUESTÃO 47

Cronometre a implementação do livro da regra dos trapézios que usa *MPI_Reduce* para diferentes números de trapézios e processos, *n* e *p*, respectivamente. Lembre-se de medir o tempo de execução ao menos 5 vezes para cada par (*n, p*).

(a) Qual critério você utilizou para escolher *n*?

Foi utilizado um n consideravelmente grande para que o tempo de execução não consiga interferir muito nos resultados, por isso o n inicial escolhido foi 50.000.000.

(b) Como os tempos mínimos se comparam aos tempos médios e medianos?

PROCESSOS ***<comm_sz>***	**TRAPÉZIOS**		
	50.000.000	**100.000.000**	**150.000.000**
1	0,865411	0,970409	1,064033
2	0,648962	0,829682	0,958068

Podemos perceber que a medida que a quantidade de trapézios aumenta isso afeta os tempos de forma negativa, ou seja, há um aumento nos tempos, mas mesmo assim os tempos entre os processos seguem o mesmo padrão de quem é maior e menor

TEMPOS	**TRAPÉZIOS**		
	50.000.000	**100.000.000**	**150.000.000**
MEDIANOS	0,648962	0,829682	0,958068
MÉDIOS	0,69161	0,86117	0,96137

(c) Quais são os *speedups*?

PROCESSOS ***<comm_sz>***	**TRAPÉZIOS**		
	50.000.000	**100.000.000**	**150.000.000**
1	1	1	1
2	1,33	1,16	1,11

(d) Quais são as eficiências?

PROCESSOS ***<comm_sz>***	**TRAPÉZIOS**		
	50.000.000	**100.000.000**	**150.000.000**
1	1	1	1
2	0,66	0,58	0,55

(e) Com base nos dados que você coletou, você diria que a regra dos trapézios é escalável?
A regra dos trapézios não poderia ser considerada escalável, pois sua eficiência não é constante.

QUESTÃO 48

Embora não conheçamos os detalhes da implementação do *MPI_Reduce*, podemos supor que ele usa uma estrutura semelhante à árvore binária que discutimos. Se for esse o caso, esperaríamos que seu tempo de execução crescesse aproximadamente à taxa de $log_2 (p)$ $(p = comm_sz)$, uma vez que existem aproximadamente $log_2 (p)$ níveis na árvore. Como o tempo de execução da regra dos trapézios serial é aproximadamente proporcional a n, o número de trapézios, e a regra dos trapézios paralela simplesmente aplica a regra serial a n/p trapézios em cada processo, com nossa suposição sobre *MPI_Reduce*, obtemos uma fórmula para o tempo de execução geral da regra dos trapézios paralela que se parece com

$$T_{parallel}(n,\ p) \approx a\frac{n}{p} + b\ log_2(p)$$

onde a e b são constantes.

Use a fórmula, os tempos que você mediu no Exercício **47** e seu programa favorito para fazer cálculos matemáticos (por exemplo, o MATLAB®) para obter uma estimativa de mínimos quadrados dos valores de a e b. Comente sobre a qualidade dos tempos de execução previstos usando a fórmula.

QUESTÃO 49
Encontre os *speedups* e as eficiências da ordenação ímpar-par paralela (`mpi_odd_even.c`).

DADOS DA MEDIANA:

PROCESSOS ***<comm_sz>***	**TAMANHO DO VETOR**			
	100.000.000	**200.000.000**	**400.000.000**	**800.000.000**
1	15,57674	32,35635	65,6125	132,8371
2	8,087162	17,34029	34,64387	69,79705

SPEEDUPS:

PROCESSOS ***<comm_sz>***	**TAMANHO DO VETOR**			
	100.000.000	**200.000.000**	**400.000.000**	**800.000.000**
1	1	1	1	1
2	1,9261	1,8660	1,8939	3,6529

EFICIÊNCIA:

PROCESSOS ***<comm_sz>***	**TAMANHO DO VETOR**			
	100.000.000	**200.000.000**	**400.000.000**	**800.000.000**
1	1	1	1	1
2	0,9631	0,9330	0,9470	1,8265

(a) O programa obtém *speedups* lineares?
Não. O *speedup* linear ideal para 2 processos seria um valor de 2.0. Conforme a tabela esses valores são bons e se aproximam de 2, mas não são perfeitamente lineares. Isso mostra que há uma pequena perda de desempenho devido à sobrecarga de comunicação do MPI. O caso com vetor de 800.000.000 elementos mostra um *speedup* anômalo de 3,6529, o que indica um possível erro na medição ou um efeito de cache extremo, não sendo representativo de um speedup linear.

(b) É escalável?
Embora escalabilidade seja a capacidade de um programa de obter ganhos de desempenho ao adicionar processadores, a aplicação em questão demonstra que não é escalável. Isso se evidencia pelo fato de que a eficiência diminui consistentemente à medida que mais processos são utilizados.

(c) É fortemente escalável?
A escalabilidade forte é avaliada mantendo o tamanho do problema fixo enquanto se aumenta o número de processos. A análise da tabela mostra que, para todos os tamanhos de problema testados, a eficiência diminui com a adição de mais processos.

Portanto, conclui-se que a aplicação não é fortemente escalável.

(d) É fracamente escalável?
A análise da escalabilidade fraca, realizada ao observar as diagonais da tabela, mostra que a eficiência não se manteve constante. Uma vez que o ideal seria manter a eficiência ao aumentar o problema e os processadores na mesma proporção, conclui-se que a aplicação não é escalável de forma fraca.

QUESTÃO 50

Modifique a ordenação ímpar-par paralela (`mpi_odd_even.c`) para que as funções *Merge* simplesmente troquem os ponteiros do vetor após encontrar os elementos menores ou maiores. Que efeito essa mudança tem no tempo de execução geral?

Tempos medianos da execução com a função memcpy

PROCESSOS ***<comm_sz>***	**TAMANHO DO VETOR**			
	100.000.000	**200.000.000**	**400.000.000**	**800.000.000**
1	15,57674	32,35635	65,6125	132,8371
2	8,087162	17,34029	34,64387	69,79705

Tempos medianos da execução com ponteiros

PROCESSOS ***<comm_sz>***	**TAMANHO DO VETOR**			
	100.000.000	**200.000.000**	**400.000.000**	**800.000.000**
1	15,4217	32,1948	65,2562	132,1109
2	8,0702	17,1584	34,0606	69,1851

A análise das tabelas revela que a estratégia de troca de ponteiros foi mais eficiente, resultando em menores tempos de execução. Este efeito foi particularmente notável em conjuntos de dados de maior escala, onde a otimização demonstrou maior impacto.

```C/C++
void Merge_low(
      int**  my_keys_pp,
      int*   recv_keys,
      int**  temp_keys_pp,
      int  local_n        ) {
   int m_i, r_i, t_i;
   int* temp;

   m_i = r_i = t_i = 0;
   while (t_i < local_n) {
      if ((*my_keys_pp)[m_i] <= recv_keys[r_i]) {
         (*temp_keys_pp)[t_i] = (*my_keys_pp)[m_i];
         t_i++; m_i++;
      } else {
         (*temp_keys_pp)[t_i] = recv_keys[r_i];
         t_i++; r_i++;
      }
   }

   temp = *my_keys_pp;
```

```
   *my_keys_pp = *temp_keys_pp;
   *temp_keys_pp = temp;
}  /* Merge_low */


void Merge_high(int** local_A_pp, int* temp_B, int** temp_C_pp, int local_n)
{
   int ai, bi, ci;
   int* temp;

   ai = local_n-1;
   bi = local_n-1;
   ci = local_n-1;
   while (ci >= 0) {
      if ((*local_A_pp)[ai] >= temp_B[bi]) {
            (*temp_C_pp)[ci] = (*local_A_pp)[ai];
            ci--; ai--;
      } else {
            (*temp_C_pp)[ci] = temp_B[bi];
            ci--; bi--;
      }
   }

   temp = *local_A_pp;
   *local_A_pp = *temp_C_pp;
   *temp_C_pp = temp;
}  /* Merge_high */
```